

Subirlo a un VPS con Docker — versión rápida

Tres cosas antes de arrancar

1. El `docker-compose.yml` del repo sólo levanta la infraestructura: **no hay ningún Dockerfile**. Los cuatro que faltan están en el paso 3.
2. **La webcam USB no existe en un VPS**. Vas a necesitar una cámara IP con RTSP alcanzable desde el servidor, o el stream de prueba del paso 8.
3. **Pedí CPU dedicada**, no compartida: el ai-worker corre YOLO al 100% sostenido y los planes burstable lo estrangulan. 4 vCPU / 8 GB para 1-2 cámaras; 8 vCPU / 16 GB para 3-4. Ubuntu Server 24.04.

1. Preparar el servidor

en el servidor

```
# Entrás como root y creás tu usuario
adduser percepta && usermod -aG sudo percepta
rsync --archive --chown=percepta:percepta ~/.ssh /home/percepta/
exit

# Ahora como percepta: firewall, Docker y swap
ssh percepta@IP_DEL_VPS

sudo apt update && sudo apt upgrade -y && sudo apt install -y ufw git curl rsync
sudo ufw allow OpenSSH && sudo ufw allow 80/tcp && sudo ufw allow 443/tcp
sudo ufw --force enable

curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER && newgrp docker

# Swap: sin esto, con 8 GB el build de Angular o de PyTorch se muere
sudo fallocate -l 4G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Ningún puerto de Percepta se abre al mundo: sólo 22, 80 y 443. Los servicios se hablan por la red interna de Docker.

2. Subir el código y los modelos

en el servidor

```
git clone git@github.com:TU_USUARIO/TU_REPO.git ~/percepta
```

Los pesos y el `.env` **no están en git** (el `.gitignore` los excluye). Copialos aparte:

desde tu PowerShell

```
scp "C:\dev\Camaras+IA\yolov8n.pt" "C:\dev\Camaras+IA\yolov8n-pose.pt" `
"C:\dev\Camaras+IA\yolov8n-seg.pt" "C:\dev\Camaras+IA\yolov8s.pt" `
percepta@IP_DEL_VPS:~/percepta/

ssh percepta@IP_DEL_VPS "mkdir -p ~/percepta/training/models"
scp "C:\dev\Camaras+IA\training\models\*" `
percepta@IP_DEL_VPS:~/percepta/training/models/
```

Si el ai-worker arranca sin los modelos no falla: levanta, no detecta nada y te volvéis loco buscando el error en otro lado. Verificá con `ls -lh ~/percepta/*.pt`.

3. Los cuatro archivos que faltan (crealos en la raíz del repo)

.dockerignore

```
node_modules
**/node_modules
.venv
.git
.turbo
.angular
.data
.runs
training/data
training/ppe/data
**/__pycache__
**/dist
```

Dockerfile.node (las cuatro APIs comparten imagen)

```
FROM node:18-bookworm-slim AS build
RUN corepack enable && corepack prepare pnpm@9.6.0 --activate
WORKDIR /app
COPY . .
RUN pnpm install --frozen-lockfile
RUN pnpm --filter @percepta/contracts build
RUN pnpm --filter @percepta/identity-service --filter @percepta/device-service \
    --filter @percepta/event-service --filter @percepta/analytics-service build

FROM node:18-bookworm-slim
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app /app
CMD ["node", "apps/identity-service/dist/main.js"]
```

Dockerfile.python (media-service y ai-worker; ~3 GB, 15 min de build)

```
FROM python:3.11-slim-bookworm
WORKDIR /app
# ffmpeg: clips H.264. build-essential: insightface y lap compilan al instalarse.
RUN apt-get update && apt-get install -y --no-install-recommends \
    ffmpeg libglib2.0-0 libgomp1 libgl1 build-essential \
    && rm -rf /var/lib/apt/lists/*
# Torch de CPU: 800 MB en vez de 2.5 GB.
RUN pip install --no-cache-dir torch==2.13.0 torchvision==0.28.0 \
    --index-url https://download.pytorch.org/whl/cpu
COPY packages/py-contracts ./packages/py-contracts
COPY apps/ai-worker ./apps/ai-worker
COPY apps/media-service ./apps/media-service
RUN pip install --no-cache-dir -e packages/py-contracts \
    && pip install --no-cache-dir -e apps/ai-worker \
    && pip install --no-cache-dir -e apps/media-service \
    && pip install --no-cache-dir "ultralytics==8.4.115" "insightface==1.0.1" \
    "onnxruntime==1.28.0" "scikit-image>=0.26" "scipy>=1.17"
COPY modules ./modules
COPY cameras.json assignments.json ./
CMD ["python", "-m", "media_service.main"]
```

Dockerfile.web

```
FROM node:18-bookworm-slim AS build
RUN corepack enable && corepack prepare pnpm@9.6.0 --activate
WORKDIR /app
COPY . .
RUN pnpm install --frozen-lockfile
RUN pnpm --filter @percepta/contracts build
RUN cd apps/web && npx ng build --configuration production

FROM nginx:1.27-alpine
COPY --from=build /app/apps/web/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

Y el nginx que **reemplaza al** proxy.conf.json de desarrollo. Sin esto el dashboard da 404 en todo:

nginx.conf

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;
    client_max_body_size 20m;

    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";

    # /api NO se reescribe. El resto sí: la barra final del proxy_pass
    # quita el prefijo, igual que el pathRewrite del proxy.conf.json.
    location /api/ { proxy_pass http://event-service:3004; }
    location /identity/ { proxy_pass http://identity-service:3001/; }
    location /device/ { proxy_pass http://device-service:3003/; }
    location /analytics/ { proxy_pass http://analytics-service:3005/; }
    location /ai/ { proxy_pass http://ai-worker:3010/; }

    # MJPEG en vivo: sin buffering y sin timeout, o se corta al minuto.
    location /media/ { proxy_pass http://media-service:3020/;
        proxy_buffering off; proxy_read_timeout 24h; }

    location / { try_files $uri $uri/ /index.html; }
}
```

4. El compose de producción

Se suma al que ya existe, no lo reemplaza.

`docker-compose.prod.yml`

```
x-node: &node
  image: percepta-node:latest
  env_file: .env
  restart: unless-stopped
  depends_on: [postgres]

x-py: &py
  image: percepta-python:latest
  env_file: .env
  restart: unless-stopped

services:
  identity-service:
    <<: *node
    build: { context: ., dockerfile: Dockerfile.node }
    command: ["node", "apps/identity-service/dist/main.js"]
  device-service:
    <<: *node
    command: ["node", "apps/device-service/dist/main.js"]
  event-service:
    <<: *node
    command: ["node", "apps/event-service/dist/main.js"]
    environment: { PORT: "3004" }
  analytics-service:
    <<: *node
    command: ["node", "apps/analytics-service/dist/main.js"]

  media-service:
    <<: *py
    build: { context: ., dockerfile: Dockerfile.python }
    command: ["python", "-m", "media_service.main"]
    environment:
      PORT: "3020"
      DEVICE_SERVICE_URL: "http://device-service:3003"
      EVIDENCE_DIR: "/app/.data/evidence"
    volumes: [evidencias:/app/.data/evidence]

  ai-worker:
    <<: *py
    command: ["python", "-m", "ai_worker.main"]
    environment:
      PORT: "3010"
      MEDIA_SERVICE_URL: "http://media-service:3020"
      EVENT_SERVICE_URL: "http://event-service:3004"
      ANALYTICS_SERVICE_URL: "http://analytics-service:3005"
      DEVICE_SERVICE_URL: "http://device-service:3003"
      AI_MODULES_PATH: "./modules"
      PIPELINE_FPS: "6"
    volumes:
      # Los pesos van montados, en la ruta exacta donde el código los busca.
      - ./yolov8n.pt:/app/yolov8n.pt:ro
      - ./yolov8n-pose.pt:/app/yolov8n-pose.pt:ro
      - ./yolov8n-seg.pt:/app/yolov8n-seg.pt:ro
      - ./yolov8s.pt:/app/yolov8s.pt:ro
      - ./training/models:/app/training/models:ro
      - insightface:/root/.insightface

  web:
    build: { context: ., dockerfile: Dockerfile.web }
    image: percepta-web:latest
    ports: ["127.0.0.1:8081:80"] # sólo local; Caddy atiende 80/443
    restart: unless-stopped

volumes: { evidencias: , insightface: }
```

5. El .env del servidor

cp .env.example .env y cambia **sólo estas líneas**: dentro de Docker cada servicio se llama por su nombre y Postgres vuelve a ser el 5432.

```
.env
NODE_ENV=production
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_PASSWORD=<<larga y única>>
DATABASE_URL=postgresql://percepta_app:<otra>@postgres:5432/percepta
DATABASE_URL_ADMIN=postgresql://percepta:<la de arriba>@postgres:5432/percepta
REDIS_URL=redis://redis:6379
RABBITMQ_PASSWORD=<<otra>>
RABBITMQ_URL=amqp://percepta:<la misma>@rabbitmq:5672
S3_ENDPOINT=http://minio:9000
S3_SECRET_KEY=<<otra>>
JWT_ACCESS_SECRET=<<openssl rand -hex 32>>
JWT_REFRESH_SECRET=<<openssl rand -hex 32>>
JWT_ACCESS_TTL=900
EVIDENCE_RETENTION_DAYS=30
COMPOSE_FILE=docker-compose.yml:docker-compose.prod.yml
```

La última línea es para no escribir los dos -f en cada comando. Después: chmod 600 .env.

6. Arrancar

en el servidor

```
cd ~/percepta
docker compose build                # 15-30 min la primera vez
docker compose up -d postgres redis rabbitmq minio
docker compose ps                  # esperá a que diga healthy

# Migraciones EN ESTE ORDEN (es el del script db:migrate, no el numérico)
docker compose exec -T postgres psql -U percepta -d percepta -v ON_ERROR_STOP=1 \
-f /migrations/0001_init.sql -f /migrations/0003_rls_hardening.sql \
-f /migrations/0007_personas.sql -f /migrations/0002_seed.sql \
-f /migrations/0004_admin_user.sql -f /migrations/0005_fall_feedback.sql \
-f /migrations/0008_control_accesos.sql -f /migrations/0009_foto_persona.sql \
-f /migrations/0010_zona_de_trabajo.sql -f /migrations/0011_kiosco.sql \
-f /migrations/0012_plano.sql -f /migrations/0013_modulo_ingreso.sql \
-f /migrations/0014_zonas.sql -f /migrations/0015_pisos.sql \
-f /migrations/0016_epp.sql -f /migrations/0017_epp_calibracion.sql

# Token de servicio (en tu PC lo genera start-all.js; acá no hay quien lo haga).
# TTL largo a propósito: si hereda los 900 s, los servicios de Python
# empiezan a dar 401 al rato de andar bien.
docker compose run --rm --no-deps -e JWT_ACCESS_TTL=31536000 \
identity-service node tools/dev-token.js service
echo 'SERVICE_TOKEN=<<el token que imprimió>>' >> .env

docker compose up -d
```

7. Dominio y HTTPS

Apuntá un registro A al IP del VPS. Hace falta de verdad: la pantalla del kiosco usa la cámara del navegador y **eso sólo funciona sobre HTTPS**.

en el servidor

```
sudo apt install -y caddy # repo oficial de Caddy
sudo nano /etc/caddy/Caddyfile
```

/etc/caddy/Caddyfile

```
percepta.tudominio.com {
    encode gzip
    @vivo path /media/*
    reverse_proxy @vivo 127.0.0.1:8081 { flush_interval -1 }
    reverse_proxy 127.0.0.1:8081
}
```

en el servidor

```
sudo systemctl reload caddy
```

8. Una cámara que el servidor pueda ver

Con cámara IP: lo mejor es Tailscale en el VPS y en tu red, y la cargás en el dashboard como

rtsp://usuario:clave@100.x.y.z:554/stream1. Abrir el 554 del router al mundo también anda, pero no lo dejes puesto.

Sin cámara todavía, para verificar el despliegue entero:

en el servidor

```
docker run -d --name mediamtx --network percepta_default bluenviron/mediamtx
docker run -d --name fuente --network percepta_default \
    -v ~/percepta/prueba.mp4:/video.mp4:ro linuxserver/ffmpeg \
    -re -stream_loop -1 -i /video.mp4 -c copy -f rtsp rtsp://mediamtx:8554/prueba
```

Y en el dashboard cargás la cámara con origen rtsp://mediamtx:8554/prueba.

9. Verificar

en el servidor

```
docker compose ps # los init salen con código 0: eso está bien

docker compose exec web sh -c '
    wget -qO- http://identity-service:3001/api/v1/health && echo
    wget -qO- http://device-service:3003/api/v1/health && echo
    wget -qO- http://event-service:3004/api/v1/health && echo
    wget -qO- http://media-service:3020/health && echo
    wget -qO- http://ai-worker:3010/health && echo '

docker stats --no-stream # el ai-worker es el que manda
```

10. Si algo falla

Síntoma	Qué hacer
404 o 502 en todo el dashboard	Barra final en el proxy_pass de /identity/, /device/, /ai/, /media/, /analytics/ — y sin barra en /api/.
La vista en vivo se corta al minuto	Falta proxy_buffering off en nginx o flush_interval -1 en Caddy.
401 en los servicios de Python después de andar bien	Venció el SERVICE_TOKEN. Regeneralo con TTL largo (paso 6) y reiniciá media-service y ai-worker.
"no se pudo abrir la fuente"	El VPS no llega a la cámara. Probá ffmpeg probe rtsp://... desde el servidor: si ahí tampoco entra, es la red.
El ai-worker levanta y no detecta nada	Faltan los .pt. docker compose exec ai-worker ls -lh /app/yolov8n.pt /app/training/models/
Todo lentísimo	CPU al tope. Bajá PIPELINE_FPS a 3, quítale módulos a la cámara, o subí el plan.
Las migraciones fallan	Las corriste en orden numérico. docker compose down -v y repetí el paso 6 con el orden exacto.

Y después

el día a día

```
docker compose logs -f ai-worker      # ver qué pasa
git pull && docker compose build && docker compose up -d  # actualizar
docker compose exec -T postgres pg_dump -U percepta percepta | gzip > bk.sql.gz
```

El usuario admin@percepta.local / percepta de la migración 0004 sigue existiendo y el dashboard entra solo con él. Está bien para probar; antes de mostrárselo a alguien de afuera, cambialo.